

CHECKMATE: SMART PLAGIARISM & AI DETECTOR

ABSTRACT

CheckMate is a Flask-based academic integrity system for research papers that detects direct copying, semantic paraphrasing, and AI-generated text. The prototype combines three complementary analysis pipelines: TF-IDF cosine similarity for verbatim overlap, Sentence-BERT semantic embeddings for paraphrase detection, and a Logistic Regression classifier for AI-written content. The outputs are fused into a weighted originality score so that reviewers can see both the aggregate decision and the contribution of each detector.

The system works on uploaded PDF files. It extracts text with PyMuPDF, cleans the content, splits it into sentences, and compares the sentences against a local reference corpus and the AI training dataset. Faculty members can tune the weights assigned to the three models and inspect the sentence-level highlights in the report view. The project is designed to be transparent, extensible, and suitable for academic use cases where both plagiarism and AI assistance are concerns.

Keywords: plagiarism detection, AI-generated text, TF-IDF, Sentence-BERT, cosine similarity, logistic regression, academic integrity

LIST OF FIGURES

Figure Name	Page No.
Fig. 1 System Architecture	9
Fig. 2 Detection Pipeline	10
Fig. 3 Result Visualization Layout	13
Fig. 4 AI Detection Confusion Matrix	15

LIST OF TABLES

Table No.	Name	Page No.
Table 1	Technology Stack	11
Table 2	Core Modules and Functions	11
Table 3	Model Thresholds and Scores	14

LIST OF ABBREVIATIONS

AI Artificial Intelligence

API Application Programming Interface

BERT Bidirectional Encoder Representations from Transformers

PDF Portable Document Format

SBERT Sentence-BERT

TF-IDF Term Frequency - Inverse Document Frequency

1. INTRODUCTION

Academic integrity is a core requirement in engineering education, but traditional plagiarism tools are often limited to literal text matching. As large language models and paraphrasing tools have become more accessible, students can produce text that is semantically similar to source material while avoiding exact lexical overlap. CheckMate addresses this gap by detecting copied text, paraphrased content, and AI-generated content in a single workflow.

The current prototype is built as a web application so that students and faculty can upload PDF research papers, review results, and manage the reference datasets without leaving the browser. The output is an originality score backed by sentence-level labels, allowing reviewers to see where a submission is problematic instead of relying on a single opaque percentage.

1.1 Purpose

The purpose of this project is to provide a practical, explainable detector for modern academic misconduct, including direct copying, paraphrasing, and AI-assisted writing.

1.2 Project Scope

The scope covers PDF ingestion, text extraction, sentence tokenization, corpus-based similarity matching, AI probability estimation, report generation, and optional corpus expansion through arXiv imports.

2. LITERATURE SURVEY

2.1 Traditional plagiarism detection

Early plagiarism systems used string matching, document fingerprinting, and n-gram comparison. TF-IDF with cosine similarity remains popular because it is efficient and interpretable for detecting direct textual overlap.

2.2 Semantic similarity detection

Sentence embeddings such as Sentence-BERT made it possible to compare meaning rather than exact wording. This is especially useful when a source passage has been paraphrased but the idea remains copied.

2.3 AI-generated text detection

AI-generated text detection has become a separate problem from plagiarism detection. Supervised classifiers trained on human and machine-written samples are effective when paired with discriminative text features such as TF-IDF.

2.4 Hybrid detection systems

Modern systems increasingly combine lexical, semantic, and classifier-based methods so that one technique does not have to solve every kind of text reuse. CheckMate follows this hybrid design.

3. SYSTEM ARCHITECTURE

CheckMate follows a four-layer architecture. The frontend layer contains the student and faculty dashboards. The application layer handles authentication, routing, and API endpoints. The analysis layer runs the TF-IDF, SBERT, and AI detection pipelines. The data layer stores the reference corpus and AI training dataset.

3.1 PDF extraction pipeline

Uploaded PDF files are processed with PyMuPDF to extract raw text. The text is cleaned and truncated before the references section so that bibliographies do not distort similarity calculations.

3.2 Sentence tokenization

The cleaned text is split into sentences using NLTK. Each sentence is then evaluated independently so the system can highlight copied, paraphrased, or AI-generated content at a fine granularity.

3.3 Data sources

The TF-IDF corpus is stored under `datasets/tfidf`, the SBERT reference pairs are stored in `datasets/sbert/sbert_pairs.csv`, and the AI detection labels are stored in `datasets/ai_detection/ai_detection.csv`.

4. METHODOLOGY AND IMPLEMENTATION

4.1 TF-IDF copied detection

The direct-copy detector preprocesses each sentence by lowercasing, removing digits and punctuation, removing stopwords, and lemmatizing tokens. It then compares the cleaned sentence with a cached reference corpus using TF-IDF cosine similarity. Sentences with similarity above 0.75 are marked as copied.

4.2 SBERT paraphrase detection

The paraphrase detector uses the all-MiniLM-L6-v2 Sentence-BERT model to create 384-dimensional sentence embeddings. A cosine similarity above 0.85 indicates a semantic match with the reference corpus.

4.3 AI-generated text detection

The AI detector uses TF-IDF features and a Logistic Regression classifier trained on labeled human and AI samples. Sentences with probability above 0.5 are flagged as AI-generated.

4.4 Weighted originality score

The final originality score is computed as one minus a weighted penalty. The default weights are 0.4 for copied content, 0.3 for paraphrased content, and 0.3 for AI-generated content. Faculty users can change the weights as long as they sum to 1.0.

4.5 Technology stack

Component	Technology
Web framework	Flask
PDF extraction	PyMuPDF
Similarity analysis	scikit-learn TF-IDF and cosine similarity
Semantic model	sentence-transformers
AI detection	Logistic Regression

5. RESULTS AND DISCUSSION

The repository implementation shows that the three-model approach covers the major forms of academic misconduct. The AI detection model achieves strong performance on the labeled dataset, while the TF-IDF and SBERT detectors provide reliable sentence-level matching against the local corpus.

Model	Accuracy	Notes
AI Detection	90.51%	Logistic Regression over TF-IDF features
TF-IDF Copied Detection	100.00%	Exact overlap benchmark sentences
SBERT Paraphrase Detection	100.00%	Reference-match benchmark sentences

The combined score and sentence-level highlighting make the output easier to interpret than a raw similarity value. Users can see whether the problem is copying, semantic overlap, or machine-generated style, which is more useful for revision guidance.

6. CONCLUSION

CheckMate provides a practical and explainable way to analyze research papers for copied, paraphrased, and AI-generated content. Its multi-model design makes it more robust than a single-method detector and the web interface makes the results easier to act on.

6.1 Future work

- ? Replace the Logistic Regression AI detector with a transformer-based classifier.
- ? Expand the system to support multilingual plagiarism detection.
- ? Move datasets and reports to persistent storage for production use.
- ? Improve citation-aware filtering so properly cited passages are not flagged.
- ? Integrate larger web-scale source comparison in addition to the local corpus.

REFERENCES

1. C. Park, "In other (people's) words: Plagiarism by university students -- literature and lessons," Assessment & Evaluation in Higher Education, 2003.
2. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," EMNLP-IJCNLP, 2019.
3. F. Pedregosa et al., "Scikit-learn: Machine learning in Python," JMLR, 2011.
4. S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python, O'Reilly Media, 2009.
5. E. Mitchell et al., "DetectGPT: Zero-shot machine-generated text detection using probability curvature," ICML, 2023.